You can also use various finder methods on a relation for performing complex calculations:

```
Client.includes("orders").where(first_name: 'Ryan', orders: { status:
'received' }).count
```

Which will execute:

```
SELECT count(DISTINCT clients.id) AS count_all FROM clients
  LEFT OUTER JOIN orders ON orders.client_id = client.id WHERE
  (clients.first_name = 'Ryan' AND orders.status = 'received')
```

20.1 Count

If you want to see how many records are in your model's table you could call `Client.count` and that will return the number. If you want to be more specific and find all the clients with their age present in the database you can use `Client.count(:age)`.

For options, please see the parent section, Calculations.

20.2 Average

If you want to see the average of a certain number in one of your tables you can call the `average`method on the class that relates to the table. This method call will look something like this:

```
Client.average("orders_count")
```

This will return a number (possibly a floating point number such as 3.14159265) representing the average value in the field.

For options, please see the parent section, Calculations.

20.3 Minimum

If you want to find the minimum value of a field in your table you can call the `minimum` method on the class that relates to the table. This method call will look something like this:

```
Client.minimum("age")
```

For options, please see the parent section, Calculations.

20.4 Maximum

If you want to find the maximum value of a field in your table you can call the `maximum` method on the class that relates to the table. This method call will look something like this:

```
Client.maximum("age")
```

For options, please see the parent section, Calculations.

20.5 Sum

If you want to find the sum of a field for all records in your table you can call the `sum` method on the class that relates to the table. This method call will look something like this:

```
Client.sum("orders_count")
```

For options, please see the parent section, Calculations.

21 Running EXPLAIN

You can run EXPLAIN on the queries triggered by relations. For example,